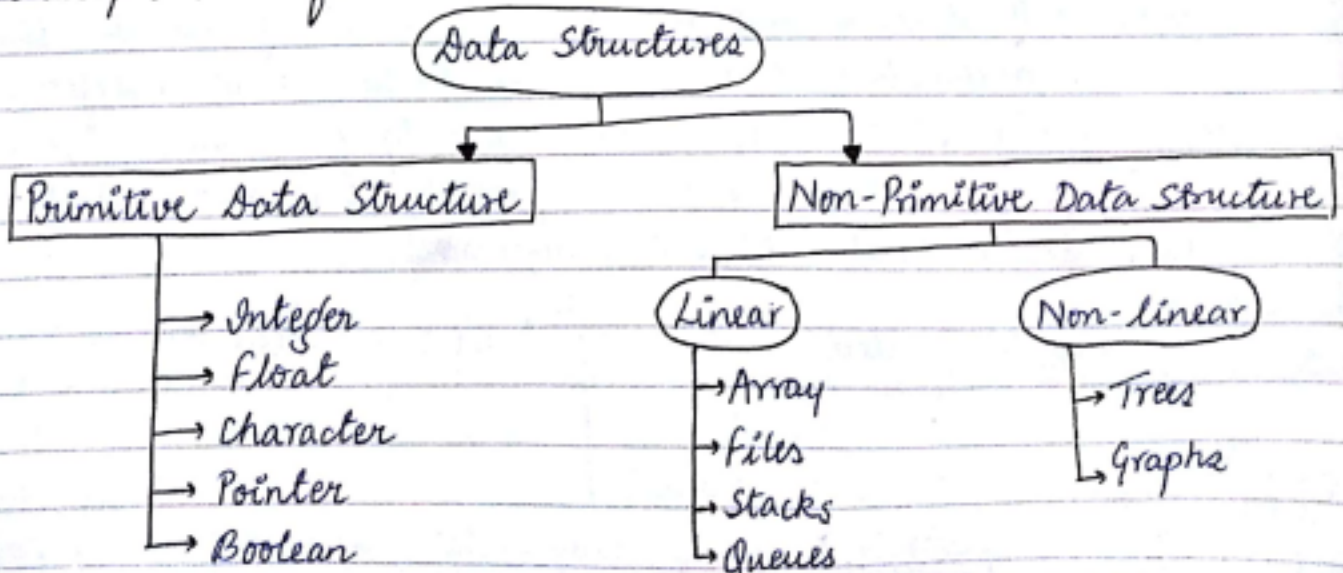


Introduction to Data Structures

- **Definition of Data structures :-** Data structures are method of representing logical relationships between individual data elements related to the solution of a given problem.
- The different models used to organise data in the main memory are collectively referred to as DATA STRUCTURES, whereas, different models used to organise data in secondary memory are collectively referred to as FILE STRUCTURE.
- DS is a structured set of variables associated with one another in different ways, co-operatively defining components in the system and capable of being operated upon in the program.
- They are basic of programming tools and the choice of data structure should provide the following :-
 - (a) Data Structures should satisfactorily represent the relationship between data elements.
 - (b) Data structures should be easy so that programmers can easily process the data.

• Classification of Data Structures :-



PRIMITIVE DATA STRUCTURES are the data structures that are directly operated upon by machine-level instructions. For ex:- integers, real, logical data, character data structure etc.

NON-PRIMITIVE DATA STRUCTURES are more complex data structures. These are derived from the primitive ones. They stress on formation of sets of homogeneous and heterogeneous data elements. For ex:- Arrays, lists, files etc.

➤ **Explanation of different Data Structures:-**

(1) **ARRAYS** → The simplest type of data structure is a linear array. An array is used to store elements of the same type. We can define array as:

<datatype> <arrayname> [size of array];

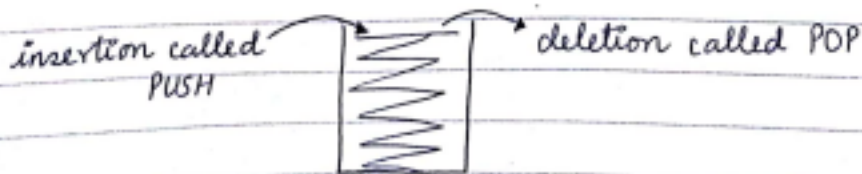
- ① Elements are stored at consecutive / continuous memory locations.
- ② Elements of array are referred by an index or sub-script.
- ③ The no. of elements is called length or size of array.

int arr[10];

2	4	6	8	10	12	14	16	18	20
---	---	---	---	----	----	----	----	----	----

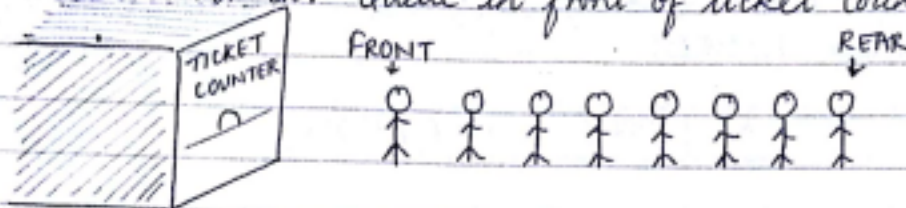
 arr[1] arr[2] - - - - - arr[10]

(2) **STACKS** → A stack is also an ordered collection of elements like array, but it has a special feature that insertion and deletion of element can be done only from one end, called TOP (Top of Stack). Due to this property it is also called LIFO (Last-In First-Out) data structure.

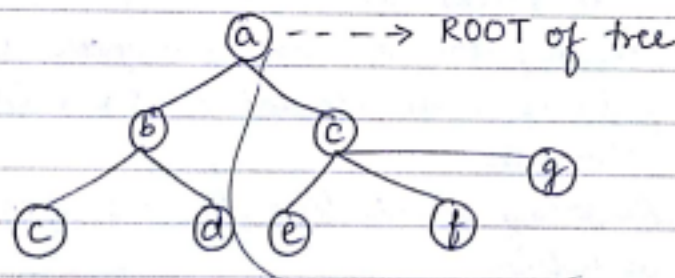


It can be implemented using array (static) as well as using pointers.

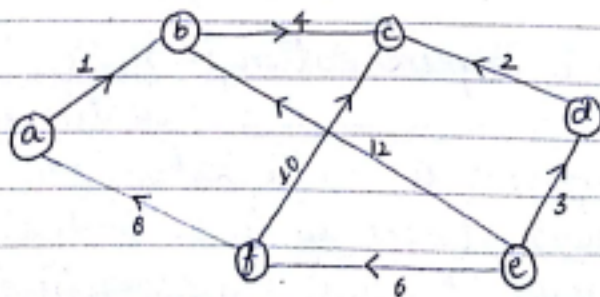
- (3) QUEUES → It is FIFO (First-In First-Out) data structures. In a queue new elements are added to the queue from one end called REAR end, and the elements are removed from another end called FRONT. For ex:- Queue in front of ticket counter.



- (4) TREES → A tree can be defined as finite set of data items called nodes. Trees are non-linear type of data structures in which data items are arranged or stored in particular sequence. It represents the hierarchical relationship between various elements. There is special data item at top called ROOT.



- (5) GRAPHS → It is mathematical non-linear data structure that is capable of representing many kind of physical structures. A Graph $G=(V,E)$ is a set of vertices V and a set of edges E .

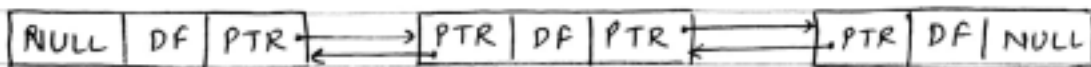
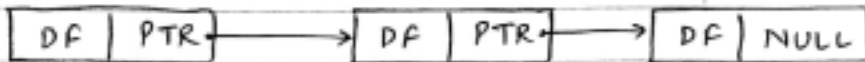


① can be weighted / un-weighted and directed / un-directed

(6) LINKED LISTS → It is a linear collection of data elements called NODES. The linear order is maintained by pointers.

Each node is divided into two or more parts:

- ① Data Field, and
- ② Pointer Field(s)



➤ Operations on Data Structures:- The various operations that can be performed on diff. data structures are:-

- (1) TRAVERSING - Accessing each element exactly once in order to process it.
- (2) SEARCHING - Finding the location of a given element.
- (3) INSERTION - Adding a new element to the existing data structure.
- (4) DELETION - Removing an existing element from the data structure.
- (5) SORTING - Arranging the elements of data structure in some logical order.
- (6) MERGING - Combining the elements of two similar data structures into a single structure.

➤ Static and Dynamic Implementation:- Static data structure is created for an input data set which is not supposed to change within the scope of the problem. When a single element is to be added or deleted, the update of a static data structure incurs significant costs. The size of data structure must be known at the time the structure

is being created. For example - an array.

It allows —

- (a) Fast access to elements
- (b) Expensive to insert / remove elements
- (c) have fixed or maximum size

Static memory allocation refers to the process of allocating memory at compile time i.e. before the associated program is executed.

The user has to pre-define the requirements of memory.

An application of this technique involves a program module declaring static data locally, such that these data are inaccessible in other modules unless references to it are passed as parameters or returned.

A single copy of static data is retained and accessible through many calls to the function in which it is declared. It has the advantage of modularizing data within a program design in the situation where these data must be retained through the run-time of the program.

But its major drawback is that if user does not use whole memory allocated at compile time, then a lot of memory is wasted.

The concept of dynamic implementation helps us to overcome the drawback of static memory allocation by allowing us to get the required chunk of memory at run-time (or we can say when the need arises). It is the one whose size can change indefinitely once it has been declared.

The size of DS does not need to be known at the time it is created. This is best suited when programmer do not know the memory requirement in advance.

It allows —

- (a) Fast updation
- (b) Slower access to elements
- (c) can have flexible size

In dynamic memory allocation, the memory storage is done during run-time of that program. There are various functions used for this

purpose like :-

(a) `malloc()` - allocates memory from heap. The prototype is:

`malloc (no. of elements * size of each element);`

The type of pointer it returns is void, which means that we can assign it any type of pointer. However, if it fails to allocate the required amount of memory, it returns a NULL.

(b) `calloc()` - This function exactly as `malloc` function except for the fact that it need two arguments against one argument of `malloc` function i.e.

`calloc (no. of elements, size of element);`

(c) `free()` - It is used to de-allocate the previously allocated memory using `malloc` or `calloc` funct^{no}. The syntax is:

`free (ptr-variable);`

(d) `realloc()` - used to resize the size of block, which is already allocated. For example:

`msg = (char *) malloc (30 * sizeof (char));`

`msg = (char *) realloc (msg, 50);`

ARRAYS

Data structure are classified as either linear or non-linear. A data structure is said to be linear if its elements form a sequence.

There are two basic way of representing linear structure :-

- (1) One way is to have the linear relationship between the elements represented by means of sequential memory locations. These linear structures are called arrays.
- (2) The other way is to have the linear relationship between the elements represented by means of pointer or links. These structures are called linked list.

Linear Arrays → A linear array is a list of finite number of homogeneous data elements.

- (a) The elements of the array are referred by an index or sub-script.
- (b) The elements of the array are stored respectively in successive memory locations.

The number of elements is called length/size of array. The length can be called as :

$$\text{length} = \text{Upper bound} - \text{Lower bound} + 1$$

Representation of linear array in memory :- let LA be a linear array in the memory of the computer (the memory is simply a sequence of addressed locations). Let us use the notation :

$\text{LOC}(\text{LA}[K]) = \text{address of the element LA}[K]$

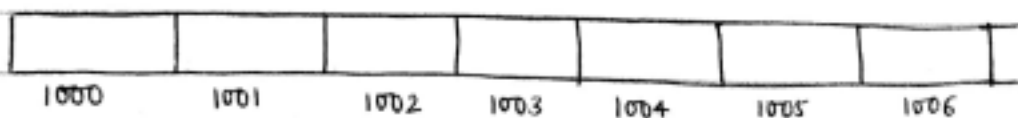


fig: Computer Memory

The elements of LA are stored in successive memory cells. Accordingly, the computer does not need to keep track of the address of every element of LA, but need to keep track of only the address of the first element of LA, denoted by Base (LA) and called the base address of LA.

Using this address Base (LA) , the computer calculates the address of any element of LA by the following formula:

$$\text{LOC (LA[K])} = \text{Base (LA)} + W (K - \text{lower bound})$$

Where, 'W' is the no. of words per memory cell for LA.

For example: Consider an array AUTO, which records the number of automobiles sold each year from 1932 through 1984. Suppose AUTO appears in memory as: Base address = 200, and $W = 4$. Calculate the address of array element for year 1965.

Solⁿ: Given: $\rightarrow$ Base Address = 200

$$W = 4$$

$$\text{Lower bound} = 1932$$

$$K = 1965$$

To calculate: $\rightarrow \text{LOC (AUTO [1965])}$

$$\begin{aligned} \text{Procedure: } \rightarrow \text{LOC (AUTO [1965])} &= \text{BA} + W (K - \text{lb}) \\ &= 200 + 4 (1965 - 1932) \\ &= 200 + 4 (33) \\ &= 200 + 132 \\ &= \underline{332} \quad \text{Ans} \end{aligned}$$

Traversing the linear array: - Let LA be an array. We want to print the contents of each element of LA. This can be accomplished by traversing LA i.e. by accessing and processing each element of LA exactly once.

Algorithm

(Traversing linear array) Here LA is a linear array with lower-bound LB and upper-bound UB. This algorithm traverses LA applying operation PROCESS to each element of LA.

- (1) [Initialize counter] Set $K := LB$.
- (2) Repeat steps 3 & 4 while $K \leq UB$.
- (3) [Visit element] Apply PROCESS to $LA[K]$.
- (4) [Increase counter] Set $K := K + 1$.
- [End of Step 2 loop]
- (5) Exit

Insertion and Deletion in linear array:- Insertion of an element at the end of a linear array can be easily done provided the memory space allocated for the array is large enough to accommodate the additional elements. On the other hand, suppose we need to insert an element in the middle, then, on the average, half of the element must be moved downwards to new locations.

Same goes for deletion operation. Deletion at end is not problematic, whereas, deletion from middle needs movement of elements.

Algorithm

(Inserting into a linear array) INSERT (LA, N, K, ITEM)

Here LA is a linear array with N elements. K is a positive integer such that $K \leq N$. This algorithm inserts an element ITEM into the K^{th} position in LA.

- (1) [Initialize counter] Set $J := N$
- (2) Repeat Steps 3 & 4 while $J \geq K$
- (3) [Move J^{th} element downwards] Set $LA[J+1] := LA[J]$
- (4) [Decrease counter] Set $J := J - 1$
- [End of Step 2 loop]

- (5) [Insert element] Set $LA[K] := ITEM$
- (6) [Reset N] Set $N := N+1$
- (7) Exit

Algorithm

(Deleting from a linear array) DELETE (LA, N, K, ITEM)

Here, LA is a linear array with N elements & K is a positive integer such that $K \leq N$. This algorithm deletes the K^{th} element from LA.

- (1) Set $ITEM := LA[K]$.
- (2) Repeat for $J = K$ to $N-1$
 [Move $(J+1)^{th}$ element upward] Set $LA[J] := LA[J+1]$
 [End of loop]
- (3) [Reset array size] Set $N := N-1$
- (4) Exit

Limitation of Array in C Programming:-

- (1) Static Data → It is static data structure as memory is allocated during compile time.
- (2) Homogeneous → Elements belonging to different data types cannot be stored in array.
- (3) Insertion and Deletion is difficult → For these operations we have to relocate the proceeding elements.
- (4) No Bound checking → C does not perform bound checking, therefore, if we try to print $(N+1)^{th}$ element of array having size N, there will be no error message, and, garbage value will be printed.
- (5) Memory shortage or wastage → If we need more space at execution time, we cannot do so. Similarly, if array of large size is defined then there is wastage of memory.

Example :- An array $X[-15 \dots 10][15 \dots 40]$ requires one byte of storage. If beginning location is 1500, determine the location of $X[15][20]$.

Solⁿ :- Given $\rightarrow$ Base Address $[X] = 1500$

To find $\rightarrow X[15][20]$

Procedure $\rightarrow$ Since no. of rows & columns is not given. So they can be calculated as:

$$\begin{aligned}\text{No. of rows, } M &= (U_r - L_r) + 1 \\ &= (10 - (-15)) + 1 \\ &= 10 + 15 + 1 \\ &= 26\end{aligned}$$

$$\begin{aligned}\text{No. of columns, } N &= (U_c - L_c) + 1 \\ &= (40 - 15) + 1 \\ &= 26\end{aligned}$$

Column-major address $\Rightarrow A[J][K] = B + W[(J - L_r) + M * (K - L_c)]$

$$B = 1500, W = 1, J = 15, K = 20, L_r = -15, L_c = 15, M = 26$$

$$\begin{aligned}\therefore X[15][20] &= 1500 + 1[(15 - (-15)) + 26 * (20 - 15)] \\ &= 1500 + 1[30 + 26 * 5] = \underline{1660}\end{aligned}$$

Row-major address $\Rightarrow A[J][K] = B + W[N * (J - L_r) + (K - L_c)]$

$$\begin{aligned}\therefore X[15][20] &= 1500 + 1[26 * (15 - (-15)) + (20 - 15)] \\ &= 1500 + 1[26 * 30 + 5] = \underline{2285}\end{aligned}$$

Sparse Matrix :- matrices with a relatively high proportion of zero entries are called sparse matrices.

$$\begin{pmatrix} 4 & & & & \\ 3 & -5 & & & \\ 1 & 0 & 6 & & \\ -7 & 8 & -1 & 3 & \\ 5 & -2 & 0 & -2 & -8 \end{pmatrix}$$

Triangular

$$\begin{pmatrix} 5 & -3 & & & \\ 1 & 4 & 3 & & \\ & 9 & -3 & 6 & \\ & & 2 & 4 & 7 \\ & & & 3 & -1 & 8 \\ & & & & 2 & -3 \end{pmatrix}$$

Tridiagonal

- First matrix where all entries above the main diagonal are zero or equivalent, where non-zero entries can only occur on or below the main diagonal is called (lower) triangular matrix.
- Second matrix, where non-zero entries can only occur on the diagonal or on elements immediately above or below the diagonal is called tri-diagonal matrix.
- Save space by storing only non-zero values.

Memory Allocation :- When allocating memory dynamically i.e. when and where required like in case of linked lists, there is some mechanism required which provides unused memory space.

Together with linked list in memory, a special list is maintained which consists of unused memory cells. This list which has its own pointer is called LIST OF AVAILABLE SPACE or FREE-STORAGE or FREE POOL.

The unused memory cells in the array will be linked together to form a linked-list AVAIL.

Garbage Collection :- Suppose some memory space becomes reusable because it is deleted from data structure. The operating system may periodically collect all the deleted space onto the AVAIL list. So the technique used is garbage collection. It takes place in two steps:-

- (i) The computer runs through all lists, tagging those cells which are currently in use, and then
- (ii) The computer runs through the memory collecting all untagged space onto free-pool.